Conv Chain 아키텍처 — 컴파일러 관점 의문사항

> **작성일**: 2026-04-08
> **작성자**: 컴파일러 담당
> **대상**: 하드웨어 아키텍처 담당
> **목적**: Conv chain 동작 모델에 대해 제가 이해한 부분을 정리하고, 컴파일러/ISA 설계에 필요한 의문사항을 공유합니다.

---

1. 내가 이해하고 있는 하드웨어 구조

1.1 전체 구성

```
┌──────────────────────────────────────────────────────────┐
│                    Shared Memory (SM)                    |
│  ┌──────────┬──────────┬──────────┬──────────────────┐  |
│  | Weights |  Input   |  Output  |  Feature Maps   |  |
│  | (상주)  | (런타임) | (런타임) |  (중간 결과)    |  |
│  └──────────┴──────────┴──────────┴──────────────────┘  |
└──────────┬──────────┬──────────┬──────────┬──────────────┘
           |          |          |          |
      ┌────┴────┐ ┌────┴────┐ ┌────┴────┐ ┌────┴────┐
      |Cluster 0||Cluster 1||Cluster 2||  ...    |  × 6 clusters
      | Core 0  || Core 2  || Core 4  ||         |
      | Core 1  || Core 3  || Core 5  ||         |  (2 cores/cluster)
      └─────────┘ └─────────┘ └─────────┘ └─────────┘
           |          |          |          |
           └──────────┴──────────┴──────────┘
                 P-core Ring / A-core Ring
```

1.2 Core 내부 구조

```
┌──────────────────────────────────────────┐
│              Core                        │
│                                          │
│  ┌──────────────────────────────────┐    │
│  │     Unified Buffer (UB)          │    │
│  │  ┌──────────┬──────────┐        │    │
│  │  │ Buffer A │ Buffer B │        │    │  ← Double Buffering (Ping-Pong)
│  │  │ (연산중) │ (DMA중)  │        │    │
│  │  └──────────┴──────────┘        │    │
│  └──────────┬───────────────────────┘    │
│             │                            │
│  ┌──────────┴──────────┐                 │
│  │   연산기            │                 │
│  │  ┌──────────┐       │                 │
│  │  │ MAC Unit │       │  ← Weight Stationary           │
│  │  │ (Conv/   │       │    Weight를 UB에 올리고         │
│  │  │  GEMM)   │       │    Input을 tiling하여 계산      │
│  │  └──────────┘       │                 │
│  │  ┌──────────┐       │                 │
│  │  │Activation│       │  ← Conv 직후 Activation 수행    │
│  │  └──────────┘       │                 │
│  └─────────────────────┘                 │
│                                          │
│  ┌──────────────────┐  ┌──────────────┐  │
│  │     Queue        │  │ Registers    │  │
│  │ (ISA records)    │  │ start        │  │
│  │                  │  │ PC           │  │
│  │                  │  │ loop_ctr     │  │
│  │                  │  │ out_buf_en   │  │
│  └──────────────────┘  └──────────────┘  │
└──────────────────────────────────────────┘
```

1.3 Weight Stationary 동작

제가 이해한 WS 동작:

```
시간 →
                                        Double Buffering으로
연산기:  [W0로 tile0 계산] [W0로 tile1 계산] ... [W0 완료] [W1로 tile0 계산] ...
                                                    ↑
UB-A:    [W0 상주 + tile 연산]                    [W1 로드 완료, 전환]
UB-B:    [W1 미리 로드 중 (DMA)]                 [W0 → W2 로드 시작]
```

- 연산 중에는 Weight가 UB의 한쪽 buffer에 상주 (Weight Stationary)
- 다음 layer의 Weight는 Double Buffering으로 다른 buffer에 미리 로드
- Layer 전환 시 buffer를 swap하여 weight reload 시간을 숨김

---

2. 내가 상상하고 있는 Conv Chain 모델

2.1 기본 아이디어

연속된 Conv layer를 core에 1:1로 배정하여 chain 형태로 처리:

```
Conv0 → Conv1 → Conv2 → Conv3 → ...
  ↓       ↓       ↓       ↓
Core0   Core1   Core2   Core3   ...
(W0상주) (W1상주) (W2상주) (W3상주)
```

2.2 이상적인 동작 시나리오 (pipeline)

만약 tile 단위로 streaming이 가능하다면:

```
시간 →     t0      t1      t2      t3      t4      t5
          ┌───────┐
Core 0:   │tile0  │tile1  │tile2  │tile3  │       │       │
          │Conv0  │Conv0  │Conv0  │Conv0  │       │       │
          └───┬───┘└───┬───┘└───┬───┘└───────┘       │       │
              │       │       │                     │       │
              ▼       ▼       ▼                     │       │
          ┌───────┐                                 │       │
Core 1:   │       │tile0' │tile1' │tile2' │tile3' │       │
          │       │Conv1  │Conv1  │Conv1  │Conv1  │       │
          └───────┘└───┬───┘└───┬───┘└───┬───┘└───────┘       │
                       │       │       │                     │
                       ▼       ▼       ▼                     │
                   ┌───────┐                                 │
Core 2:            │       │       │tile0''│tile1''│tile2''│
                   │       │       │Conv2  │Conv2  │Conv2  │
                   └───────┘       └───────┘└───────┘└───────┘

→ 이상적: 모든 core가 동시에 일하며, 전체 latency 감소
```

2.3 현실적 동작 시나리오 (layer 단위)

Accumulation 때문에 layer가 완료되어야 다음 layer가 시작 가능하면:

```
시간 →     t0      t1      t2      t3      t4      t5
          ┌──────────────────────────┐
Core 0:   │  Conv0 전체 계산         │  idle   │  idle   │
          │  (모든 tile 누적)        │         │         │
          └──────────────────────────┘         │         │
                     │ FM 전달                  │         │
                     ▼                          │         │
                   ┌──────────────────────────┐ │         │
Core 1:  idle      │  Conv1 전체 계산         │  idle     │
                   │  (모든 tile 누적)        │           │
                   └──────────────────────────┘           │
                              │ FM 전달                   │
                              ▼                           │
                            ┌──────────────────────────┐  │
Core 2:  idle    idle       │  Conv2 전체 계산         │  │
                            │  (모든 tile 누적)        │  │
                            └──────────────────────────┘  │

→ 현실: core가 대부분 idle. pipeline 효과 없음.
→ 이득은 weight reload 회피 + SM 왕복 감소뿐.
```

---

3. 의문사항

의문 1: Accumulation과 Conv Chain의 양립

Convolution의 출력은 모든 input channel에 대한 누적이 완료되어야 유효합니다.

```
Output[c_out, h, w] = Σ  Input[c_in, h+kh, w+kw] × Weight[c_out, c_in, kh, kw]
                      c_in
                      kh,kw
                      ─────
                      이 합이 끝나야 output 1개 완성

그 후:
  → bias 더하기
  → requantize (i32 → i8, scale 적용)
  → activation (SiLU 등)
  → 최종 i8 output
  ──────────────────────
  여기까지 끝나야 다음 Conv의 input으로 사용 가능
```

질문: Conv chain에서, Core 0이 Conv0의 **일부 tile**만 완료했을 때 Core 1이 Conv1을 시작할 수 있는 시나리오가 있나요?

- 만약 **있다면**: tile 단위 동기화가 필요합니다 (§3.의문3 참조)
- 만약 **없다면** (Conv0 전체가 끝나야 Conv1 시작): §2.3의 "현실적 시나리오"가 됩니다

의문 2: Channel Tiling과 Accumulation

하드웨어 문서에 "Input tiling 우선순위: Channel → H" 라고 되어 있습니다.

Channel을 자르면 **partial sum 누적**이 필요합니다:

```
Conv (C_in=128, C_out=64, kernel=3×3)

Channel tile 0: partial += Input[c=0~31]   × Weight[:, 0~31, :, :]
Channel tile 1: partial += Input[c=32~63]  × Weight[:, 32~63, :, :]
Channel tile 2: partial += Input[c=64~95]  × Weight[:, 64~95, :, :]
Channel tile 3: partial += Input[c=96~127] × Weight[:, 96~127, :, :]
                ────────
                4번 다 끝나야 output 완성 (i32 → requant → activation → i8)
```

```
UB 내부 흐름:
┌──────────────────────────────────────────────────────────┐
│ Channel tile 0            Channel tile 1                 │
│ ┌──────────┐              ┌──────────┐                   │
│ │W[0:31]   │  MAC ──→     │W[32:63]  │  MAC ──→          │
│ │I[0:31]   │  partial     │I[32:63]  │  partial          │
│ └──────────┘  (i32)       └──────────┘  += (i32)         │
│                                                          │
│ Channel tile 2            Channel tile 3                 │
│ ┌──────────┐              ┌──────────┐                   │
│ │W[64:95]  │  MAC ──→     │W[96:127] │  MAC ──→          │
│ │I[64:95]  │  partial     │I[96:127] │  최종 누적      |
│ └──────────┘  += (i32)    └──────────┘  (i32)            │
│                                                          │
│              → requantize → activation → i8 output       │
└──────────────────────────────────────────────────────────┘
```

질문: 이 channel tiling + accumulation 과정에서:
- partial sum (i32)은 **UB에 상주**하나요? 아니면 별도 accumulation buffer가 있나요?
- Weight 교체(tile별)와 Input 교체(tile별)가 Double Buffering으로 처리되나요?
- 모든 channel tile 누적이 끝난 후 requant+activation이 수행되는 것이 맞나요?

의문 3: Core 간 동기화 메커니즘

Conv chain에서 Core 0의 결과를 Core 1이 받으려면 **동기화**가 필요합니다.

두 가지 모델을 생각하고 있는데, 어느 쪽인지 확인이 필요합니다.

Model A: Event-driven (컴파일러가 DMA + 동기화를 명시적으로 지시)

```
Core 0 Queue:                      Core 1 Queue:
┌──────────────────┐               ┌──────────────────┐
│ layer_start:     │               │                  │
│   DMA(SM→UB, W0)│               │ WAIT(Core0)      │ ← Core 0 완료 대기
│                  │               │                  │
│ layer_run:       │               │ layer_start:     │
│   Conv0(UB주소) │              │   DMA(SM→UB, W1) │
│                  │               │                  │
│ layer_end:       │               │ layer_run:       │
│   DMA(UB→SM)    │              │   Conv1(UB주소)  │
│   SIGNAL(Core1) │──event──→    │                  │
│                  │               │ layer_end:       │
│ 다음 layer...    │              │   DMA(UB→SM)     │
└──────────────────┘               │   SIGNAL(Core2) │
                                   └──────────────────┘
```

- 컴파일러가 모든 DMA와 동기화를 ISA에 인코딩
- **장점**: 정확한 제어, 디버깅 용이
- **단점**: Queue 크기 부담 (tile 단위라면 record 수 폭증)
- **필요한 것**: event/signal 레지스터, wait instruction

Model B: Core 자율 (SM 주소만 주고 내부는 core가 관리)

```
Core 0 Queue:                      Core 1 Queue:
┌──────────────────┐               ┌──────────────────┐
│ layer_run:       │               │ layer_run:       │
│   Conv0          │               │   Conv1          │
│   (SM addrs)     │               │   (SM addrs)     │
│   core가 알아서:│              │  core가 알아서: │
│   SM→UB fetch    │               │   SM→UB fetch    │
│   tiling         │               │   ???            │
│   compute        │               │                  │
│   UB→SM store    │               │                  │
└──────────────────┘               └──────────────────┘

                                        ↑
                                 Core 0이 끝났는지
                                 어떻게 아나?
```

- Core 내부에서 SM→UB→연산→UB→SM을 자율 처리
- **장점**: Queue 극소화, 컴파일러 단순
- **단점**: Core 간 동기화 방법이 없음
- **질문**: Core 1이 Conv1을 시작해도 되는지 어떻게 판단하나요?

⚠️ Model B에서의 핵심 문제: tile 단위 chain 동기화

Model B(Core 자율)에서 Conv chain을 tile 단위로 streaming하려는 경우를 생각해봅니다:

```
Core 0 (Conv0, 자율 동작):          Core 1 (Conv1, 자율 동작):
┌──────────────────────┐            ┌──────────────────────┐
│ SM에서 input 가져옴  │            │                      │
│ 내부적으로 tiling    │            │  Conv0의 tile이      │
│                      │            │  끝났는지 어떻게     │
│ tile 0 계산 완료 ────┼───???????──→│  알 수 있나?         │
│ tile 1 계산 중...    │            │                      │
│                      │            │  Core 0 내부의       │
│                      │            │  tiling 진행 상황을  │
│                      │            │  Core 1은 알 수 없음 │
└──────────────────────┘            └──────────────────────┘
```

이것은 컴파일러가 해결할 수 있는 문제가 아닙니다. 그 이유:

```
┌──────────────────────────────────────────────────────────────┐
│  "컴파일러가 알아서 해줄 수 있지 않나?"에 대한 답        │
│                                                              │
│  컴파일러는 실행 전에 바이너리를 생성하는 정적 도구임.    │
│  실행 시점의 동적 상태를 제어할 수 없음.                  │
│                                                              │
│  컴파일러가 할 수 있는 것:                                 │
│    ✓ Queue에 명령어를 배치                                 │
│    ✓ DMA 주소와 크기를 사전 계산                           │
│    ✓ 동기화 instruction을 삽입 (HW가 지원하면)            │
│                                                              │
│  컴파일러가 할 수 없는 것:                                 │
│    ✗ Core 0의 tile 0이 "지금" 끝났는지 감지                │
│    ✗ Core 1에게 "지금 시작해도 돼"라고 실시간 전달        │
│    ✗ DMA 지연이나 메모리 충돌을 실시간 대응               │
│                                                              │
│  결론: 동기화는 하드웨어 메커니즘이 반드시 필요.          │
│        컴파일러는 그 메커니즘을 "사용"하는 명령어를       │
│        생성할 수 있을 뿐, 메커니즘 자체를 대체할 수 없음.│
└──────────────────────────────────────────────────────────────┘
```

구체적으로:

```
Model B + tile 단위 chain을 원한다면:

  하드웨어가 제공해야 하는 것:
  ┌──────────────────────────────────────┐
  │ Core 0 tile 완료 → 하드웨어 signal    │
  │                        ↓              │
  │              signal 전파 (Ring? 레지스터?)
  │                        ↓              │
  │ Core 1이 signal을 감지 → 연산 시작     │
  └──────────────────────────────────────┘

  이것이 없으면:
  ┌──────────────────────────────────────┐
  │ Core 0 tile 완료                      │
  │       ↓                               │
  │ Core 1은 아무것도 모름                 │
  │       ↓                               │
  │ 두 가지 선택:                          │
  │   (a) 타이밍으로 추측 → 위험 (의문5)    │
  │   (b) layer 전체 끝날때까지 대기 → 느림 │
  └──────────────────────────────────────┘
```

정리: Model B에서 core가 내부 tiling을 자율적으로 하면, **tile 단위 진행 상황이 외부에서 보이지 않습니다**. 따라서:
- tile 단위 chain → 하드웨어가 tile 완료 signal 메커니즘을 제공해야 함
- layer 단위 chain → 기존 output_buffer_enable 레지스터로 가능하나 pipeline 효과 없음
- **컴파일러는 HW가 제공하는 동기화 메커니즘에 맞춰 ISA를 생성할 수 있지만, 메커니즘이 없으면 아무것도 할 수 없음**

Model A와 B의 비교

```
                      Model A              Model B
                   (Event-driven)       (Core 자율)
                  ┌──────────────┐    ┌──────────────┐
Queue 사용량      │ 많음 (DMA+   │    │ 적음 (Conv   │
                  │ Signal+Wait) │    │ 명령만)      │
                  ├──────────────┤    ├──────────────┤
동기화            │ 명시적       │    │ ???          │
                  │ (레지스터)   │    │ (방법 없음?) │
                  ├──────────────┤    ├──────────────┤
컴파일러 역할     │ DMA+동기화   │    │ SM 주소만    │
                  │ 모두 생성    │    │              │
                  ├──────────────┤    ├──────────────┤
디버깅            │ 용이         │    │ 어려움       │
                  │ (모든 것이   │    │ (core 내부   │
                  │ ISA에 기록)  │    │ 블랙박스)    │
                  └──────────────┘    └──────────────┘
```

질문: 현재 어떤 모델로 설계하고 있나요? 혹시 두 모델의 **혼합**을 생각하고 있나요?

의문 4: "layer"의 범위

ISA에서 `layer_start` / `layer_run` / `layer_end`가 있는데, 여기서 "layer"가 의미하는 범위가 중요합니다.

```
의미 A: "layer" = 모델의 Conv layer 1개 전체

  layer_start → DMA (전체 weight + 전체 input)
  layer_run   → Conv (전체 출력 계산)
  layer_end   → DMA (전체 output 저장)

  문제: 데이터가 UB에 안 들어감 (weight + FM > UB)


의미 B: "layer" = tiling된 1개 tile 연산

  layer_start → DMA (weight slice + input tile)
  layer_run   → Conv (1 tile 계산)
  layer_end   → DMA (output tile 저장)

  문제: tile 수만큼 instruction record 필요 → Queue 크기 부담


의미 C: "layer" = Conv layer 1개이지만, core가 내부적으로 tiling

  layer_start → (없음 또는 최소 DMA)
  layer_run   → Conv(SM주소) → core가 알아서 tiling
  layer_end   → (core가 알아서 SM에 저장)

  문제: 동기화 메커니즘 필요 (Model B의 문제)
```

```
┌──────────────────────────────────────────────────────────┐
│  Queue Record 수 비교                                    │
│                                                          │
│  의미 A: 83 layers × 3 records = ~249 records            │  ← Queue에 들어감
│  의미 B: 83 layers × 25 tiles × 3 = ~6225 records        │  ← Queue 초과!
│  의미 C: 83 layers × 1 record = ~83 records              │  ← 여유
│                                                          │
│  Queue 최대: 512 records (128KB / 256B)                  │
└──────────────────────────────────────────────────────────┘
```

질문: ISA에서 "1개 instruction layer"는 위의 A, B, C 중 어느 것인가요?

의문 5: Core 간 동기화 레지스터

Model A(Event-driven)을 채택한다면, 동기화를 위한 레지스터가 필요합니다.

```
현재 정의된 레지스터:
  - start register          (런타임 → core 시작)
  - program counter         (Queue 내 위치)
  - loop counter            (반복 횟수)
  - output buffer enable    (추론 완료 신호)

추가로 필요할 수 있는 것:
  - event flag register     (core 간 signal/wait)
  - DMA status register     (DMA 완료 확인)
```

질문:
- Core 간 동기화(signal/wait)를 레지스터로 구현하는 것을 고려하고 있나요?
- 아니면 순수 타이밍(clock cycle 계산)으로 동기화를 맡기는 것을 생각하고 있나요?

순수 타이밍 동기화의 위험

"컴파일러가 clock cycle을 계산해서 맞춰줄 수 있지 않나?"라는 생각이 있을 수 있습니다.

```
시나리오: 컴파일러가 Conv0 = 1000 clock으로 계산하여,
         Core 1의 Queue에 "1001 clock에 시작"이라고 인코딩

정상 동작:
  ┌──────────────────────────────────────────────┐
  │ clock: 0        500       1000      1001     │
  │                                              │
  │ Core 0: [====Conv0 계산====]  완료           │
  │                                    ↓         │
  │ Core 1:                          [시작] OK!  │
  └──────────────────────────────────────────────┘

DMA 버스 충돌 발생 시:
  ┌──────────────────────────────────────────────┐
  │ clock: 0        500       1000 1001    1050  │
  │                                              │
  │ Core 0: [====Conv0 계산==][DMA대기][완료]    │
  │                              ↑    ↓          │
  │                            버스 충돌!        │
  │                                              │
  │ Core 1:                    [시작]            │
  │                              ↑               │
  │                      아직 Conv0이 안 끝남!   │
  │                      → 잘못된 데이터로 계산  │
  └──────────────────────────────────────────────┘
```

타이밍이 어긋나는 원인들 (컴파일러가 예측할 수 없음):

```
┌──────────────────────────────────────────────────────────┐
│ 1. DMA 버스 충돌                                          │
│    - 6개 DMA 버스를 12 core가 공유                         │
│    - 동시 접근 시 대기 발생 → 지연                          │
│                                                          │
│ 2. SM 뱅크 충돌                                           │
│    - 같은 뱅크에 여러 core가 동시 접근                      │
│    - 뱅크 arbiter 대기 → 지연                              │
│                                                          │
│ 3. Ring 전송 충돌                                         │
│    - 12-core ring에서 여러 전송이 동시 발생                 │
│    - hop 수에 따른 가변 latency                            │
│                                                          │
│ 4. Double Buffering 전환 타이밍                            │
│    - Weight 로드가 연산보다 느릴 수 있음                    │
│    - buffer swap 대기 → 지연                               │
│                                                          │
│ 이 중 어느 하나라도 발생하면 타이밍 동기화 실패.            │
│ 컴파일러는 이런 동적 충돌을 예측할 수 없음.                 │
└──────────────────────────────────────────────────────────┘
```

의문 3(Model B)과의 연결:

```
Model B (Core 자율) + 순수 타이밍 동기화:

  컴파일러: "Conv0은 대략 1000 clock이니까
             Core 1은 1001에 시작하도록 설정해줄게"

  현실: Core 0이 내부적으로 tiling을 어떻게 하는지
        컴파일러는 모름 (Core 자율이므로).
        따라서 clock 수 자체를 예측할 수 없음.

  → Model B에서 타이밍 동기화는 이중으로 불가능:
    1) Core 내부 동작을 컴파일러가 모르므로 clock 예측 불가
    2) 예측하더라도 동적 충돌로 어긋남
```

결론: 동기화는 반드시 **하드웨어 레지스터(event flag)** 기반이어야 하며, 컴파일러는 해당 레지스터를 사용하는 **ISA instruction(wait/signal)**을 생성하는 역할만 할 수 있습니다.

의문 6: Ring vs SM 경유 — 데이터 전달 경로

```
방안 A: Ring 직접 전달 (UB → Ring → UB)
┌─────────┐      Ring Bus      ┌─────────┐
│ Core 0 │ ──────────────→ │ Core 1 │
│  UB    │   FM 직접 전달  │  UB    │
└─────────┘                    └─────────┘
장점: SM 대역폭 소비 없음
단점: Ring 대역폭이 FM 크기를 감당?
      (첫 layer: 320×320×16 = 1.6MB)


방안 B: SM 경유 (UB → SM → UB)
┌─────────┐                ┌─────────┐
│ Core 0 │                │ Core 1 │
│  UB    │                │  UB    │
└────┬────┘                └────┬────┘
     │     ┌──────────────┐     │
     └───→│   SM         │←────┘
          │ (FM 저장)    │
          └──────────────┘
장점: 기존 DMA 인프라 재사용
단점: SM 대역폭 공유 (12 core가 동시 접근)


방안 C: DMA 경유 (UB → DMA → UB)
┌─────────┐      DMA Bus       ┌─────────┐
│ Core 0 │ ─────────────→ │ Core 1 │
│  UB    │  비동기 전송   │  UB    │
└─────────┘                    └─────────┘
장점: 비동기, 연산과 전송 겹침 가능
단점: DMA 버스 6개 공유 → 충돌 가능
```

질문: 어느 방안을 기본으로 생각하고 있나요? 혹시 상황에 따라 혼합 사용?

---

4. 요약 — 답이 필요한 항목

#	질문	선택지	영향받는 것
1	Conv chain에서 tile 단위 streaming 가능?	가능 / 불가능 (layer 단위)	pipeline 효과 유무
2	Channel tiling 시 partial sum은 어디에?	UB 상주 / 별도 버퍼	UB 크기 계산
3	Core 간 동기화 모델은?	Model A (event) / B (자율) / 혼합	ISA 구조, Queue 크기
4	ISA "layer"의 범위는?	전체 layer / tile / core 자율	Queue 사용량, 컴파일러 역할
5	동기화용 레지스터 추가 예정?	있음 / 타이밍으로 처리	안정성, ISA 확장
6	Core 간 데이터 경로는?	Ring / SM 경유 / DMA	대역폭, latency

이 항목들이 확정되면 컴파일러의 ISA 생성 로직과 시뮬레이터 설계를 진행할 수 있습니다.